07 - 训练全流程面试 50 题

> 从预训练到 RLHF 对齐的完整训练链路，每道题深入 MiniMind 的具体实现，帮助你全面掌握 LLM 训练的每个环节。

Q1: Next Token Prediction 的训练目标是什么？Loss 函数是什么？

标准答案：

Next Token Prediction（NTP）是自回归语言模型的核心训练目标：给定前面所有 token，预测下一个 token。形式化地，最大化序列的对数似然：

$$\max_\theta \sum_{t=1}^{N} \log P(x_t \mid x_1, x_2, \ldots, x_{t-1}; \theta)$$

等价于最小化交叉熵损失（Cross-Entropy Loss）：

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^{N} \log P(x_t \mid x_1, \ldots, x_{t-1}; \theta)$$

其中 $P(x_t)$ 是模型在 token x_t 上的预测概率。交叉熵损失衡量模型预测分布与真实分布（one-hot）之间的差距。训练采用 Teacher Forcing：一次性输入整个序列，通过 Causal Mask 确保位置 t 只看到 $\leq t$ 的 token，并行计算所有位置的 loss。

MiniMind 中的实现：

```
logits = model(input_ids)
loss = F.cross_entropy(
    logits.view(-1, vocab_size),
    labels.view(-1),
    ignore_index=-100  # 忽略 padding 位置
)
```

输入和标签有一个 token 的偏移：`input_ids = token_ids[:-1]`，`labels = token_ids[1:]`。预训练时所有 token 都参与 loss 计算（loss_mask 全 1，padding 除外）。

追问方向： - 交叉熵损失和 KL 散度有什么关系？ - Perplexity 和交叉熵 loss 的关系是什么？（PPL = $e^{\mathcal{L}}$）

Q2: 预训练数据有哪些来源？如何做数据清洗和去重？

标准答案：

预训练数据通常来自以下来源：网页爬取（CommonCrawl）、书籍（Books3）、百科（Wikipedia）、代码仓库（GitHub）、学术论文（ArXiv）、新闻等。

数据清洗流程包括： 1. **语言检测**：过滤非目标语言的文本 2. **质量过滤**：基于 Perplexity 过滤（用小语言模型打分，去掉 PPL 过高/过低的样本） 3. **启发式规则**：过滤过短/过长、特殊字符比例过高、重复率过高的文本 4. **安全过滤**：去除有毒内容、个人隐私信息 5. **去重**：精确去重（哈希）+ 模糊去重（MinHash + LSH） 6. **格式标准化**：统一编码格式、换行符等

数据清洗在实际工程中通常占整个项目 60% 以上的时间。

MiniMind 中的实现：

MiniMind 的预训练数据使用 JSONL 格式（每行一个 JSON 对象 `{"text": "..."}`），已经过清洗和去重处理。提供两个版本： - `pretrain_t2t_mini.jsonl`（1.2GB）：快速实验用，约 1.21h 训练完 - `pretrain_t2t.jsonl`（10GB）：完整训练用，约 10h 训练完

追问方向： - Scaling Law 对数据量有什么要求？ - 数据清洗不充分对模型有什么影响？

Q3: MinHash/SimHash 去重的原理是什么？

标准答案：

MinHash 是一种用于估计两个集合 Jaccard 相似度的技术： 1. 将文本转化为 n-gram 集合（如 5-gram） 2. 对集合应用多个不同的哈希函数，取每个哈希函数下的最小值，形成签名向量 3. 两个文档的签名向量中相同位置值相同的概率等于它们的 Jaccard 相似度 4. 配合 LSH（Locality-Sensitive Hashing）进行候选对筛选，将相似文档映射到同一桶中，避免全量两两比较

SimHash 是另一种近似去重方法： 1. 将文本分词后，对每个词计算哈希值，按照 TF-IDF 权重加权 2. 每一位上正负权重求和，正数取 1、负数取 0，生成一个固定长度的指纹 3. 两个文档的 SimHash 指纹的汉明距离（Hamming Distance）越小，文本越相似 4. 通常汉明距离 ≤ 3 认为是近似重复

MinHash 基于集合相似度，适合文本片段级去重；SimHash 基于加权哈希，适合文档级去重。

MiniMind 中的实现：

MiniMind 的预训练数据已经过去重处理。文档 L11 中提到数据清洗步骤包括"去重：删除完全相同或高度相似的样本（MinHash、SimHash）"。在实际工程中，MinHash + LSH 是处理大规模预训练数据去重的主流方案。

追问方向： - MinHash 的哈希函数个数如何选择？ - LSH 的桶数（band）和行数（row）如何影响去重的精度和召回？

Q4: 什么是 Scaling Laws？它如何指导模型设计？

标准答案：

Scaling Laws（缩放法则）由 Kaplan et al. (2020) 和 Hoffmann et al. (2022, Chinchilla) 提出，描述了 LLM 性能与三个因素的幂律关系：

$L(N, D, C) \propto N^{-\alpha_N} + D^{-\alpha_D} + C^{-\alpha_C}$

- **N**：模型参数量
- **D**：训练数据量（token 数）
- **C**：计算预算（FLOPs）

核心结论： 1. **模型越大、数据越多、训练越久，loss 越低**，且遵循可预测的幂律曲线 2. **Chinchilla 法则**：最优训练时，数据 token 数应约为参数量的 20 倍（$D \approx 20N$） 3. 给定固定计算预算，存在最优的模型大小和数据量组合

MiniMind 中的实现：

MiniMind 是 64M 参数的小模型，按 Chinchilla 法则，最优数据量约 $64M \times 20 = 1.28B$ tokens。MiniMind 的全量预训练数据约 10GB 文本，大致在这个量级。模型设计选择了 `dim=768, n_layers=8, n_heads=16, vocab_size=6400`，在小模型尺度下平衡了参数量和训练开销。

追问方向： - Chinchilla 法则和原始 Scaling Laws 有什么不同？ - 对于小模型（<1B），Scaling Laws 还成立吗？

Q5: 预训练的学习率调度策略有哪些？MiniMind 用的是什么？

标准答案：

常见的学习率调度策略包括： 1. **线性衰减（Linear Decay）**：从最大学习率线性降到最小值 2. **余弦退火（Cosine Annealing）**：按余弦曲线平滑衰减，业界主流 3. **多步衰减（Step Decay）**：在固定步数处乘以衰减因子 4. **WSD（Warmup-Stable-Decay）**：warmup → 稳

定 → 衰减三段式，最近流行 5. **逆平方根（Inverse Sqrt）**：$\eta = \eta_0 / \sqrt{t}$，Transformer 原论文使用

绝大多数 LLM 预训练使用 **Warmup + Cosine Annealing** 的组合：先线性增长到最大学习率，再按余弦曲线衰减到最小学习率。

MiniMind 中的实现：

MiniMind 使用自定义的 Warmup + Cosine Annealing 调度器：

```
def get_lr(current_step, warmup_steps, max_steps, max_lr, min_lr):
    if current_step < warmup_steps:
        return max_lr * current_step / warmup_steps
    if current_step > max_steps:
        return min_lr
    decay_ratio = (current_step - warmup_steps) / (max_steps - warmup_steps)
    coeff = 0.5 * (1.0 + math.cos(math.pi * decay_ratio))
    return min_lr + coeff * (max_lr - min_lr)
```

预训练最大学习率 `max_lr=5e-4`，通过手动设置 `optimizer.param_groups` 中的 `lr` 来应用。

追问方向： - 为什么不使用 PyTorch 内置的 CosineAnnealingLR？ - Warmup 步数通常占总步数的多少比例？

Q6: 余弦退火（Cosine Annealing）的公式是什么？为什么不让 lr 降到 0？

标准答案：

余弦退火的公式为：

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\frac{t - t_w}{T - t_w} \cdot \pi))$$

其中 $\eta_{\max}$ 为最大学习率，$\eta_{\min}$ 为最小学习率，t_w 为 warmup 步数，T 为总训练步数。

不让 lr 降到 0 的原因： 1. **保持学习能力**：如果 lr=0，模型完全停止学习，训练末期遇到的数据完全浪费 2. **防止陷入局部最优**：微小的学习率保持参数的轻微扰动，有助于跳出浅层局部最优 3. **实践经验**：通常 $\eta_{\min} = 0.1 \times \eta_{\max}$，即最终学习率为峰值的 10%

MiniMind 中的实现：

MiniMind 的余弦退火公式可等价表示为：

$$lr = max_lr \times (0.1 + 0.45 \times (1 + \cos(\pi \times \frac{step}{total})))$$

其中 `min_lr` 不为 0，确保训练全程模型都在学习。MiniMind 的 `get_lr` 函数在 warmup 后返回 `min_lr + coeff * (max_lr - min_lr)`，`min_lr > 0`。

追问方向： - 余弦退火和线性衰减在实际 loss 曲线上有什么差异？ - 有没有重启余弦退火（Cosine Annealing with Warm Restarts）？适用场景是什么？

Q7: Warmup 的作用是什么？为什么训练初期要用小学习率？

标准答案：

Warmup 是指在训练初期，将学习率从 0（或极小值）线性增长到目标学习率的过程。其作用包括：

1. **防止参数震荡**：训练初期模型参数随机初始化，梯度方向不可靠，大学习率会导致参数剧烈震荡甚至 NaN
2. **稳定 Adam 的二阶矩估计**：Adam 优化器的二阶矩 v_t 需要几百步来"热身"，初期估计不准确。如果直接用大 lr，$\frac{m_t}{\sqrt{v_t}}$ 的有效步长可能极大
3. **LayerNorm 的初始化**：训练初期 LayerNorm/RMSNorm 的统计量不稳定，需要温和的梯度逐步调整
4. **Loss Landscape 探索**：小步长帮助模型先找到相对合理的参数空间，再加速学习

通常 warmup 步数为总训练步数的 1%-5%。

MiniMind 中的实现：

```
if current_step < warmup_steps:
    return max_lr * current_step / warmup_steps
```

采用线性 warmup：`lr = max_lr * step / warmup_steps`。这是 LLM 预训练的标准配置，简单有效。

追问方向： - 如果不用 warmup，直接开始训练会怎样？ - 有没有除了线性 warmup 之外的其他 warmup 策略？

Q8: 梯度累积的原理是什么？和增大 batch_size 有什么区别？

标准答案：

梯度累积（Gradient Accumulation）是在多个 mini-batch 上累加梯度后才执行一次参数更新的技术。等效 batch_size = micro_batch_size × accumulation_steps × num_gpus。

```
for step, batch in enumerate(dataloader):
    loss = model(batch) / accumulation_steps  # 除以累积步数
    loss.backward()  # 梯度累加到 .grad 上
    if (step + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

和直接增大 batch_size 的区别： 1. **数学上等价**：梯度是线性算子，分多次求和等于一次求和 2. **BN 统计量不同**：但 Transformer 用 LayerNorm/RMSNorm，不受影响 3. **显存差异**：梯度累积每次只需加载一个 micro-batch 的激活值，显存占用远小于大 batch 4. **吞吐量差异**：梯度累积需要多次前向/反向传播，throughput 稍低 5. **关键注意**：loss 必须除以 `accumulation_steps`，否则累积梯度的量级会被放大 N 倍

MiniMind 中的实现：

MiniMind 预训练使用 `batch_size=32, accumulation_steps=8`，等效 batch_size = 256。训练循环中在累积步结束时才做 `scaler.step(optimizer)` 和 `optimizer.zero_grad()`，并在 backward 之前将 loss 除以 `accumulation_steps`。

追问方向： - 梯度累积和分布式 DDP 可以同时使用吗？如何计算等效 batch_size？ - 梯度累积对 learning rate 有什么隐含影响？

Q9: 混合精度训练（AMP）的原理是什么？bf16 和 fp16 的区别？

标准答案：

混合精度训练在前向和反向传播时使用低精度浮点（FP16/BF16），在参数存储和更新时保留 FP32（master weight），从而： 1. **减少显存占用**：FP16/BF16 是 2 字节，FP32 是 4 字节，显存减半 2. **加速计算**：GPU Tensor Core 对半精度有 2-4 倍加速 3. **保持精度**：关键操作（权重更新、损失计算）仍用 FP32

FP16 vs BF16 对比：

特性	FP16	BF16
指数位	5 位	8 位
尾数位	10 位	7 位

特性	FP16	BF16
数值范围	±65504（较小）	与 FP32 相同（±3.4e38）
精度	较高（~3-4 位有效数字）	较低（~2-3 位有效数字）
溢出风险	高，需要 Loss Scaling	低，通常不需要
硬件支持	A100/3090 等	A100/3090 等（较新 GPU）

BF16 的数值范围与 FP32 相同，几乎不会发生上溢（overflow），是目前 LLM 训练的首选。FP16 精度更高但范围小，容易溢出，必须配合 GradScaler。

MiniMind 中的实现：

MiniMind 使用 BF16 混合精度训练：

```
with autocast(dtype=torch.bfloat16):
    logits = model(input_ids)
    loss = compute_loss(logits, labels)
```

使用 `torch.cuda.amp.autocast` 自动管理精度转换，BF16 场景下通常可以省略 GradScaler。

追问方向： - 哪些操作必须保持 FP32？（softmax、layer norm、loss 计算等） - TF32 是什么？和 BF16 有什么区别？

Q10: GradScaler 在 fp16 训练中的作用是什么？

标准答案：

GradScaler（梯度缩放器）是 FP16 混合精度训练中防止梯度下溢（underflow）的关键组件。

问题背景：FP16 能表示的最小正数约为 6×10^{-8}，而深度网络的梯度值常常小于这个阈值，导致梯度变成 0（下溢），模型无法更新。

GradScaler 的工作原理： 1. **前向传播后**：将 loss 乘以一个较大的 scale factor（如 65536），放大 loss 2. **反向传播**：由于链式法则，所有梯度也被放大相同倍数，避免下溢 3. **参数更新前**：`scaler.unscale_(optimizer)` 将梯度缩小回原始量级 4. **动态调整**：如果检测到梯度中有 `inf` 或 `NaN`（溢出），跳过本次更新并减小 scale factor；否则尝试增大 scale factor

```
scaler = GradScaler()
with autocast(dtype=torch.float16):
    loss = model(input)
scaler.scale(loss).backward()
```

```
scaler.unscale_(optimizer)
clip_grad_norm_(model.parameters(), max_norm=1.0)
scaler.step(optimizer)
scaler.update()
```

BF16 不需要 GradScaler：BF16 的数值范围与 FP32 相同，梯度几乎不会下溢，因此通常可以省略 scaler。

MiniMind 中的实现：

MiniMind 使用 BF16 训练，但代码中仍保留了 `scaler` 的框架以兼容 FP16 场景。训练循环中使用 `scaler.scale(loss).backward()` → `scaler.unscale_(optimizer)` → `clip_grad_norm_` → `scaler.step(optimizer)` → `scaler.update()` 的完整流程。当使用 BF16 时，scaler 实际上不做缩放操作。

追问方向： - GradScaler 的 scale factor 是如何动态调整的？ - 如果梯度同时有下溢和上溢的风险怎么办？

Q11: 预训练时如何构造训练样本？BOS/EOS token 如何处理？

标准答案：

预训练样本的构造流程： 1. 从 JSONL 文件读取文本 2. 用 Tokenizer 将文本编码为 token id 序列 3. 截断到 `max_seq_len` 4. 构造 input 和 label：输入为序列前 n-1 个 token，标签为后 n-1 个 token（右移一位）

```
原始序列:  [BOS, A, B, C, D, E, EOS]
输入 (x):  [BOS, A, B, C, D, E]
标签 (y):  [A, B, C, D, E, EOS]
```

BOS/EOS token 的处理： - **BOS（Begin of Sequence, `<s>`）**：标记序列开始，让模型知道这是一个新文档的起点 - **EOS（End of Sequence, `</s>`）**：标记序列结束，模型学会在适当时候停止生成 - 预训练时通常在文本开头加 BOS、末尾加 EOS - 多个文档拼接时，EOS 作为文档分隔符

MiniMind 中的实现：

MiniMind 的分词器定义了 `<s>` 和 `</s>` 作为 BOS/EOS token。预训练数据处理中：

```
token_ids = tokenizer.encode(text)
input_ids = token_ids[:-1]
```

```
labels = token_ids[1:]
loss_mask = [1] * len(labels)  # 预训练全部参与 loss
```

SFT 阶段使用 ChatML 风格的特殊 token `<|im_start|>` 和 `<|im_end|>` 替代传统的 BOS/EOS 来标记对话边界。

追问方向： - 多个文档拼接为一个训练样本时，Attention Mask 需要怎样处理？ - 为什么要做文档拼接（packing）？不拼接有什么问题？

Q12: 什么是 torch.compile？它是如何加速训练的？

标准答案：

`torch.compile` 是 PyTorch 2.0 引入的即时编译（JIT Compilation）功能，通过将 Python 动态图编译为优化后的计算图来加速训练和推理。

加速原理： 1. **算子融合（Operator Fusion）**：将多个小算子合并为一个大 kernel，减少 GPU kernel 启动开销和显存读写次数。例如将 `LayerNorm` 的减均值、除方差、缩放三步合为一个 kernel 2. **内存优化**：减少中间变量的显存分配，优化张量布局 3. **图优化**：消除冗余计算、常量折叠 4. **代码生成**：使用 Triton 或 Inductor 后端生成高效的 GPU kernel

使用方式：

```
model = torch.compile(model)
```

注意事项： - 首次运行会有编译开销（几十秒到几分钟） - 对动态 shape 支持有限，需要 `dynamic=True` - 不是所有操作都能被编译优化 - 加速效果因模型结构而异，通常有 10%-50% 的提升

MiniMind 中的实现：

MiniMind 的文档和代码中未显式使用 `torch.compile`。作为一个教学项目，MiniMind 保持了代码的可读性和调试友好性。在实际生产环境中，可以在训练脚本中加入 `model = torch.compile(model, mode='reduce-overhead')` 来获取加速，尤其对于重复执行的训练循环效果显著。

追问方向： - torch.compile 的 `mode` 参数（default/reduce-overhead/max-autotune）各有什么区别？ - torch.compile 和 TorchScript 有什么区别？

Q13: SFT 和预训练在训练目标上有什么区别？

标准答案：

维度	预训练（Pretrain）	SFT（Supervised Fine-Tuning）
训练目标	学习语言的通用知识和统计规律	学习遵循指令和对话的能力
数据格式	纯文本 `{"text": "..."}`	多轮对话 `{"conversations": [...]}`
Loss 计算	所有 token 都计算 loss	只计算 assistant 回复部分的 loss
数据来源	网页、书籍、百科等无标注文本	高质量指令-回复对（人工标注或强模型生成）
数据量	海量（TB 级）	相对少（万到百万条）
学习率	较大（如 5e-4）	较小（通常是预训练的 1/10~1/50）
轮数	通常 1 epoch	1-3 epoch
输出行为	文本续写/补全	理解指令、结构化回答

核心区别在于 **Loss Mask**：预训练学的是"所有文本的分布"，SFT 学的是"如何回答"。SFT 不改变模型的知识，而是教它正确的"表达方式"。

MiniMind 中的实现：

- 预训练：`train_pretrain.py`，`lr=5e-4`，`batch_size=32`，`max_seq_len=340`
- SFT：`train_full_sft.py`，`lr=1e-5`（比预训练小 50 倍），`batch_size=16`，`max_seq_len=768`
- SFT 数据 14GB 包含大量知识注入（mid-training 概念），模糊了预训练和 SFT 的边界

追问方向： - 什么是 mid-training？它和传统 SFT 有什么区别？ - SFT 后的模型能否"忘记"预训练学到的知识？

Q14: 为什么 SFT 只计算 assistant 部分的 loss？如何实现 loss_mask？

标准答案：

为什么只计算 assistant 部分？ 1. **训练目标**：SFT 的目标是训练模型"如何回答"，而非"如何提问"。计算 prompt 的 loss 等于教模型生成用户的话，偏离目标 2. **梯度信号纯度**：如果也计算 prompt loss，大量梯度信号被"预测 prompt token"占据，稀释了"生成好回复"的学习信号 3. **避免过拟合 prompt**：SFT 数据中的 prompt 可能来自特定模板，计算其 loss 会让模型过拟合到特

定提问方式 4. **信息论角度**：prompt 部分的信息模型通过预训练已经学会了，assistant 回复才是 SFT 阶段的新监督信号

loss_mask 的实现：

```
loss = F.cross_entropy(
    logits.view(-1, vocab_size),
    labels.view(-1),
    reduction='none'  # 不做 reduce，得到逐 token loss
)
loss = (loss * loss_mask.view(-1)).sum() / loss_mask.sum()  # mask 后求平均
```

MiniMind 中的实现：

MiniMind 在 `train_full_sft.py` 中按角色构建 loss_mask：

```
for turn in conversations:
    role = turn['role']
    turn_tokens = tokenizer.encode(f"<|im_start|>{role}\n{content}<|im_end|>\n")
    if role == 'assistant':
        mask = [0] * role_prefix_len + [1] * content_len + [1]  # 包含 <|im_end|>
    else:
        mask = [0] * len(turn_tokens)
```

assistant 的角色前缀（`<|im_start|>assistant\n`）mask 为 0，内容和结束符 mask 为 1。

追问方向： - `<|im_end|>` 要不要计入 loss？为什么？ - 如果把 loss_mask 搞反了（只计算 prompt 的 loss），模型会学成什么样？

Q15: Chat Template 是什么？为什么需要统一格式？

标准答案：

Chat Template 是将多轮对话结构化为模型输入的格式模板，使用特殊 token 标记角色和对话边界。常见格式包括 ChatML、Alpaca、Llama-2 Chat 等。

为什么需要统一格式？ 1. **角色分离**：模型需要知道哪部分是 system、user、assistant，才能区分"谁在说话" 2. **格式统一**：训练和推理必须使用完全一致的模板，否则模型行为不可预期 3. **控制生成**：特殊 token（如 `<|im_end|>`）帮助模型知道何时停止生成 4. **构建 loss_mask**：根据模板的特殊 token 自动定位 assistant 部分 5. **多轮支持**：自然地支持多轮对话的上下文拼接

注意：如果训练和推理的 Chat Template 不一致，模型表现会**急剧下降**——这是工程中常见的坑。

MiniMind 中的实现：

MiniMind 使用类似 ChatML（OpenAI）的格式：

```
<|im_start|>system
你是一个有用的助手。<|im_end|>
<|im_start|>user
什么是机器学习？<|im_end|>
<|im_start|>assistant
机器学习是人工智能的一个子领域...<|im_end|>
```

推理时的模板函数：

```
def apply_chat_template(messages):
    result = ""
    for msg in messages:
        result += f"<|im_start|>{msg['role']}\n{msg['content']}<|im_end|>\n"
    result += "<|im_start|>assistant\n"
    return result
```

追问方向： - ChatML 格式和 Llama-2 Chat 格式有什么区别？ - 如果用户在输入中故意包含特殊 token（如 `<|im_end|>`），会发生什么？如何防御？

Q16: MiniMind 的 Chat Template 格式是什么样的？

标准答案：

MiniMind 采用 ChatML 风格的对话模板，使用 `<|im_start|>` 和 `<|im_end|>` 作为轮次边界标记。

完整格式示例：

```
<|im_start|>system
你是一个有用的助手。<|im_end|>
<|im_start|>user
什么是量子力学？<|im_end|>
<|im_start|>assistant
量子力学是物理学的一个基本理论...<|im_end|>
```

特殊 Token 含义： | Token | 含义 | |-------|------| | `<|im_start|>` | 一轮对话开始（im = inner monologue） | | `<|im_end|>` | 一轮对话结束 | | `system` | 系统指令角色 | | `user` | 用户角色 | | `assistant` | 助手角色 |

MiniMind 中的实现：

模板在训练和推理中必须一致。训练时，SFT 数据按照此格式拼接并构建 loss_mask；推理时，用户输入也按同样格式包装后送入模型，最后以 `<|im_start|>assistant\n` 作为生成提示。MiniMind 词表（6400）中包含了这些特殊 token 的编码。

追问方向： - 推理时为什么要在末尾加 `<|im_start|>assistant\n`？ - 如果换用其他 Chat Template（如 Alpaca 格式），需要重新 SFT 吗？

Q17: 多轮对话的 SFT 数据如何构造？

标准答案：

多轮对话的 SFT 数据需要将所有轮次拼接成一个完整序列，并为每一轮的 assistant 回复构建 loss_mask：

```
第1轮 System:    <|im_start|>system\n你是助手<|im_end|>\n     → mask=0
第1轮 User:      <|im_start|>user\n你好<|im_end|>\n           → mask=0
第1轮 Assistant:  <|im_start|>assistant\n你好！<|im_end|>\n    → 内容 mask=1
第2轮 User:      <|im_start|>user\n天气如何<|im_end|>\n        → mask=0
第2轮 Assistant:  <|im_start|>assistant\n今天晴朗<|im_end|>\n  → 内容 mask=1
```

关键要点： 1. 所有轮次按序拼接，角色标记和内容一起编码 2. 每轮 assistant 的回复内容 mask=1，其他全部 mask=0 3. 如果总长度超过 `max_seq_len`，需要截断（通常保留最近的上下文） 4. 后续轮次的 assistant 可以看到前面所有轮次的内容（Causal Mask 保证因果性） 5. 每一轮的 `<|im_end|>` 也要计入对应角色的 mask

MiniMind 中的实现：

MiniMind 的 SFT 数据集 `sft_t2t.jsonl`（14GB）包含多轮对话数据，格式为 `{"conversations": [{"role": "...", "content": "..."}, ...]}`。数据处理时遍历 conversations 列表，按角色构建 loss_mask，截断到 `max_seq_len=768`。

追问方向： - 多轮对话中间的 user 消息被 mask 掉了，模型怎么理解上下文的？ - 截断时从头部截还是尾部截？为什么？

Q18: SFT 的学习率为什么要比预训练小一个量级？

标准答案：

SFT 学习率通常是预训练的 1/10 到 1/50，原因如下：

1. **保护预训练知识**：预训练模型已经学到了丰富的语言知识和世界知识。大学习率会破坏这些知识，导致灾难性遗忘
2. **参数空间已经合理**：预训练后的参数已经处于一个好的参数空间中，只需要小幅调整来适配对话格式
3. **SFT 数据量小**：SFT 数据量远小于预训练数据，大学习率容易过拟合
4. **微调 vs 从头训练**：微调本质上是在已有知识的基础上做精细调整，步子太大容易"跑偏"
5. **loss 已经较低**：SFT 初始 loss 比预训练低得多（因为从预训练 checkpoint 开始），需要更小的步长来精细优化

MiniMind 中的实现：

- 预训练学习率：`lr = 5e-4`
- SFT 学习率：`lr = 1e-5`（小了 50 倍）

这个比例关系在业界是典型的。有些框架也支持分层学习率（底层更小、顶层更大），但 MiniMind 使用统一的 `1e-5`。

追问方向： - 如果 SFT 学习率和预训练一样大，会发生什么？ - 有没有自动搜索最优 SFT 学习率的方法？

Q19: 什么是灾难性遗忘？SFT 时如何缓解？

标准答案：

灾难性遗忘（Catastrophic Forgetting）是指模型在学习新任务时，严重遗忘之前学过的知识。在 LLM 中，SFT 时可能出现：模型学会了对话能力，但丢失了预训练阶段获得的某些常识或语言能力。

缓解策略： 1. **使用小学习率**：限制参数更新幅度，保护已有知识（最常用） 2. **LoRA 等参数高效方法**：只更新少量参数，冻结大部分预训练权重，天然防遗忘 3. **数据混合**：在 SFT 数据中混入部分预训练数据（replay），保持原有能力 4. **正则化**：EWC（弹性权重巩固）等方法对重要参数施加更强的正则化 5. **控制训练轮数**：通常 SFT 1-3 epoch 即可，过多会加重遗忘 6. **权重衰减**：适当的 weight decay 也有正则化作用

MiniMind 中的实现：

MiniMind 通过以下方式缓解灾难性遗忘： - SFT 学习率 `1e-5`（预训练的 1/50） - 提供 LoRA 微调方案（只训练约 2% 参数） - 14GB SFT 数据本身包含大量知识（mid-training），兼顾了知识注入和格式对齐 - 支持全参微调和 LoRA 微调两种方案，用户可根据场景选择

追问方向： - LoRA 为什么能天然防止灾难性遗忘？ - 如何定量评估模型是否发生了灾难性遗忘？

Q20: SFT 数据质量和数量哪个更重要？

标准答案：

数据质量远比数据数量重要，这在 LLM 社区已成共识。

LIMA 论文的实验：仅用 1000 条精心挑选的高质量 SFT 数据，就能让 LLaMA-65B 达到接近 GPT-4 的对话能力，证明 SFT 的核心价值在于"格式对齐"而非知识注入。

低质量数据的危害： - 错误的事实会被模型学到 - 风格不一致会让模型输出混乱 - 过于简短或冗长的回答会影响模型的回答长度偏好 - 格式混乱（Markdown、纯文本、HTML 混杂）降低输出质量

高质量数据的特征： - 回答正确、信息丰富 - 风格统一 - 覆盖多样的问题类型 - 格式规范

MiniMind 中的实现：

MiniMind 使用了两种 SFT 数据策略： - 小规模 `sft_t2t_mini.jsonl` （1.6GB）：快速实验 - 全量 `sft_t2t.jsonl` （14GB）：包含 Tool Call 数据，不仅做格式对齐还注入大量知识 - SFT 数据的高质量回答来自 Qwen3、DeepSeek R1 等强模型生成（黑盒蒸馏）

追问方向： - 如何自动化地评估 SFT 数据的质量？ - 少量高质量数据 vs 大量中等质量数据，在不同模型规模下的取舍？

Q21: System Prompt 的作用是什么？MiniMind 如何随机添加 system prompt？

标准答案：

System Prompt 的作用： 1. **设定角色和行为**：如"你是一个有用的助手"，约束模型的回复风格 2. **注入领域知识**：如"你是一个医疗专家"，引导模型在特定领域回答 3. **安全约束**：如"不要生成有害内容"，增强模型的安全性 4. **格式控制**：如"请用 JSON 格式回答"，控制输出格式

随机添加 system prompt 的意义： - 训练时随机添加/不添加 system prompt，让模型学会在有无 system prompt 两种场景下都能正常工作 - 避免模型过度依赖 system prompt - 增加训练数据的多样性

MiniMind 中的实现：

MiniMind 在 SFT 数据处理中，system prompt 作为对话的第一轮（`role: "system"`）参与 chat template 格式化。训练时有一定概率随机添加或省略 system prompt： - 当有 system prompt 时：`<|im_start|>system\n你是一个有用的助手。<|im_end|>\n` 作为序列开头 - 当省略时：直接从 user 轮次开始 - 无论是否有 system prompt，其内容都不参与 loss 计算（mask=0）

追问方向： - 推理时不加 system prompt，模型的表现会有多大差异？ - 如何让模型更好地遵循 system prompt 中的指令？

Q22: 什么是 thinking token（`<think>` 标签）？MiniMind 如何控制其出现比例？

标准答案：

Thinking token（思考标签）是一种让模型先进行"内部思考"再给出最终答案的机制，受 Chain-of-Thought 和 DeepSeek R1 的启发。在回复中，`<think>...</think>` 包裹的内容是模型的推理过程，之后才是面向用户的正式回答。

作用： 1. **提升推理能力**：显式的推理链条帮助模型进行多步推理 2. **可解释性**：用户可以看到模型的思考过程 3. **质量控制**：思考过程可以帮助模型组织更好的答案

控制出现比例的意义： - 不是所有问题都需要深度思考（简单问候不需要 `<think>`） - 过多的 thinking token 会增加生成延迟和 token 消耗 - 需要在训练数据中控制 thinking 样本的比例

MiniMind 中的实现：

MiniMind 的词表（6400）中包含了 `<think>` 和 `</think>` 特殊 token。在推理阶段，通过 `open_thinking` 参数控制是否启用思考模式。当启用时，模型先生成思考内容（在 `<think>...</think>` 标签内），然后输出正式回答。在 SFT 训练数据中，包含一定比例的带思考过程的样本，让模型学会何时需要思考、何时可以直接回答。

追问方向： - thinking token 参与 loss 计算吗？为什么？ - 如何训练模型自己判断何时需要 `<think>`、何时不需要？

Q23: LoRA 的核心思想是什么？数学原理？

标准答案：

核心假设：预训练模型在适配下游任务时，权重的变化量 ΔW 是**低秩**的（intrinsic dimensionality 很低）。因此可以将 ΔW 分解为两个低秩矩阵的乘积。

数学公式：

$h = (W_0 + \Delta W)x = W_0 x + BAx$

其中： - $W_0 \in \mathbb{R}^{d \times d}$：冻结的预训练权重 - $A \in \mathbb{R}^{r \times d}$：降维矩阵 - $B \in \mathbb{R}^{d \times r}$：升维矩阵 - $r \ll d$：秩，远小于原始维度

带缩放因子的完整公式：

$h = W_0 x + \frac{\alpha}{r} \cdot BAx$

参数量对比： - 全量微调：d^2 个参数 - LoRA（r=8, d=768）：$2 \times r \times d = 2 \times 8 \times 768 = 12,288$，仅占原始的 2.08%

MiniMind 中的实现：

MiniMind **纯手写** LoRA，不依赖 peft 库：

```
class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=8, alpha=16):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features, bias=False)
        self.linear.weight.requires_grad = False  # 冻结原始权重
        self.lora_A = nn.Parameter(torch.randn(r, in_features) * 0.01)
        self.lora_B = nn.Parameter(torch.zeros(out_features, r))
        self.scaling = alpha / r

    def forward(self, x):
        base_output = self.linear(x)
        lora_output = (x @ self.lora_A.T @ self.lora_B.T) * self.scaling
        return base_output + lora_output
```

通过 `convert_to_lora(base_model, r=8, alpha=16)` 替换模型中的线性层。

追问方向： - LoRA 的 intrinsic dimensionality 假设有什么理论支持？ - 为什么 LoRA 比 Adapter 方法更受欢迎？

Q24: 为什么 A 矩阵高斯初始化、B 矩阵零初始化？

标准答案：

LoRA 中 A 矩阵用高斯随机初始化（$\mathcal{N}(0, 0.01^2)$），B 矩阵用零初始化，原因是：

B = 0 的核心目的：确保训练开始时 $BA = 0$，LoRA 旁路不会改变原始模型的输出。 1. **保留预训练知识**：初始时模型行为与原始模型完全一致，从一个好的起点开始微调 2. **训练稳定性**：如果 A 和 B 都随机初始化，BA 的初始输出是非零的随机噪声，会立即破坏原始模型的性能 3. **渐进式学习**：训练过程中 B 从零逐渐学到有意义的值，实现平滑过渡

为什么不是 A=0, B=random? - 数学上两种方式等价（都能保证 $BA = 0$） - 但 A 随机初始化可以提供不同的降维基底，让训练更容易（类似于打破对称性）

如果 A 和 B 都初始化为零? - 梯度也为零（因为 LoRA 的梯度依赖于 A 和 B 本身），模型永远无法更新，陷入"死区"

MiniMind 中的实现:

```
self.lora_A = nn.Parameter(torch.randn(r, in_features) * 0.01)  # 高斯初始化
self.lora_B = nn.Parameter(torch.zeros(out_features, r))         # 零初始化
```

乘以 0.01 是为了让 A 的初始值较小，避免训练初期 LoRA 旁路的梯度过大。

追问方向: - 如果使用 Kaiming 初始化 A 矩阵，效果会有变化吗? - A 矩阵的初始化方差（0.01 vs 0.1）对训练有什么影响?

Q25: LoRA 的秩 r 如何选择? 太大或太小有什么影响?

标准答案:

r 的选择指南: - $r = 1$~4：极简任务（如情感分类），参数最少 - $r = 4$~8：大多数场景，尤其是与预训练域接近的任务 - $r = 8$~16：中等复杂任务（指令微调、领域适配） - $r = 16$~64：复杂任务或与预训练域差距大的场景

r 太小的影响: - 表达能力不足，无法充分捕获任务所需的权重变化 - 适配效果差，和全量微调差距大

r 太大的影响: - 参数量增加，失去 LoRA 的效率优势 - 接近全量微调，可能过拟合（尤其数据量少时） - 计算和显存开销增大

影响因素: 1. 任务复杂度：复杂任务需要更大 r 2. 数据量：数据多可用大 r（不容易过拟合） 3. 模型规模：大模型的 intrinsic dimension 更低，可用相对更小的 r 4. 最终需要在验证集上对比不同 r 的效果

原始论文实验表明，$r = 4$ 在大多数 NLP 任务上就能达到接近全量微调的效果。

MiniMind 中的实现:

MiniMind 默认使用 `r=8, alpha=16` （ `scaling = alpha/r = 2.0` ）。对于 64M 参数的小模型来说，r = 8~16 是合理选择。简历式描述中也提到 r = 16 的配置。

追问方向： - r 和 alpha 的关系是什么？alpha 通常怎么设？ - 有没有自适应选择 r 的方法？（如 AdaLoRA）

Q26: LoRA 权重如何合并回原模型？

标准答案：

LoRA 训练完成后，可以将 LoRA 权重永久合并到原始权重中，合并后模型和普通模型完全相同，推理时**零额外开销**。

合并公式：

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} \cdot BA$$

其中 $\frac{\alpha}{r}$ 是缩放因子，BA 是 LoRA 的增量矩阵。

合并的优势： 1. 推理时无额外计算（不需要走 LoRA 旁路） 2. 模型结构和普通模型完全一致，无需特殊推理框架 3. 模型文件大小和普通模型相同

不合并的优势： 1. 可以动态切换不同的 LoRA（多任务服务） 2. 基础模型只需存一份，多个 LoRA 适配器各几 MB

MiniMind 中的实现：

MiniMind 在 `scripts/convert_model.py` 中实现了 LoRA 权重合并：

```
def merge_lora_weights(model):
    for module in model.modules():
        if isinstance(module, LoRALinear):
            delta_w = (module.lora_B @ module.lora_A) * module.scaling
            module.linear.weight.data += delta_w
    return model
```

合并后可以直接 `torch.save(model.state_dict(), 'merged_model.pt')` 保存为普通模型。

追问方向： - 合并后还能恢复出 LoRA 权重吗？ - 多个 LoRA 能否同时合并？有什么注意事项？

Q27: QLoRA 和 LoRA 有什么区别？

标准答案：

QLoRA（Quantized LoRA）在 LoRA 基础上进一步优化，将冻结的原始权重**量化到 4-bit**，大幅降低显存占用。

维度	LoRA	QLoRA
冻结权重精度	FP16/FP32	**4-bit（NF4）**
LoRA 权重精度	FP16	FP16（反向传播时用 BF16）
显存占用	中等	极低
适用规模	中小模型	**大模型（7B+ 在消费级 GPU 上微调）**

QLoRA 的三大关键技术： 1. **NF4 量化**：一种专为正态分布设计的 4-bit 数据类型，信息损失最小 2. **双重量化（Double Quantization）**：连量化常数也量化，额外节省约 0.37 bits/参数 3. **分页优化器（Paged Optimizers）**：利用 CPU 内存处理优化器状态的峰值，避免 OOM

QLoRA 让你可以在消费级 GPU（24GB）上微调 7B 甚至 13B 的模型，且效果接近全参微调。

MiniMind 中的实现：

MiniMind 本身只有 64M 参数，显存充裕，不需要 QLoRA。但 QLoRA 的思路值得理解，对于将 MiniMind 的方法迁移到更大模型时非常实用。MiniMind 的手写 LoRA 实现是理解 QLoRA 原理的基础。

追问方向： - NF4 和普通 INT4 量化有什么区别？ - QLoRA 的精度损失有多大？

Q28: MiniMind 的 LoRA 是如何手写实现的？（不依赖 peft 库）

标准答案：

MiniMind 的 LoRA 完全从零实现，核心分为三个部分：

1. LoRALinear 层定义（`model/model_lora.py`）：

```
class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=8, alpha=16):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features, bias=False)
        self.linear.weight.requires_grad = False  # 冻结
        self.lora_A = nn.Parameter(torch.randn(r, in_features) * 0.01)
```

```
        self.lora_B = nn.Parameter(torch.zeros(out_features, r))
        self.scaling = alpha / r

    def forward(self, x):
        return self.linear(x) + (x @ self.lora_A.T @ self.lora_B.T) * self.scaling
```

2. 模型转换（`convert_to_lora`）：遍历模型所有线性层，替换为 LoRALinear，加载预训练权重到 `self.linear`，冻结所有非 LoRA 参数：

```
for name, param in lora_model.named_parameters():
    if 'lora' not in name:
        param.requires_grad = False
```

3. 权重合并（`scripts/convert_model.py`）：训练后将 $\frac{\alpha}{r} \cdot BA$ 加回原始权重。

不用 peft 的好处： - 代码完全可控，便于学习和调试 - 不依赖第三方库，减少兼容性问题 - 可以自由修改 LoRA 的行为（如只对特定层加 LoRA）

MiniMind 中的实现：

训练脚本 `trainer/train_lora.py` 的关键流程：加载预训练 checkpoint → `convert_to_lora(base_model, r=8, alpha=16)` → 冻结非 LoRA 参数 → 用 AdamW 优化 LoRA 参数（`lr=1e-4`）→ 训练完成后合并权重。可训练参数通常占总参数的约 2%。

追问方向： - 手写 LoRA 和 peft 库的实现在效果上有区别吗？ - 如何扩展这个实现以支持 DoRA 或 rsLoRA?

Q29: RLHF 的完整流程是什么？为什么 SFT 后还需要对齐？

标准答案：

RLHF（Reinforcement Learning from Human Feedback）的经典三步流程（InstructGPT）：

```
Step 1: SFT          → 用指令数据训练模型，使其学会对话格式
Step 2: 训练 RM      → 用偏好数据训练奖励模型，学会评估回答质量
Step 3: PPO 训练     → 用 RL 优化策略模型，使其输出高奖励的回答
```

为什么 SFT 后还需要对齐？

SFT 本质上是**模仿学习**——让模型学会"像训练数据那样说话"，但存在局限： 1. **无法区分好坏**：SFT 只有"标准答案"，模型不知道什么是"好回答"vs"更好的回答" 2. **表面模仿**：模型可能学到表

面模式而非真正理解偏好 3. **安全性不足**：SFT 无法有效教模型"什么不该说"

对齐的三个维度：Helpful（有帮助）、Honest（诚实）、Harmless（无害）。

MiniMind 中的实现：

MiniMind 实现了完整的对齐训练链路： - **DPO**（`train_dpo.py`）：直接从偏好数据学习，不需要 RM，使用 `dpo.jsonl` - **PPO/GRPO**（`train_rl.py`）：使用 InternLM2-1.8B-Reward 作为奖励模型，`rlaif.jsonl` 提供 prompt

追问方向： - RLHF 中的 Step 2 和 Step 3 能否合并？（DPO 就是一种合并方案） - Constitutional AI（CAI）和 RLHF 有什么区别？

Q30: DPO 的数学推导是什么？它如何避免训练奖励模型？

标准答案：

推导过程：

Step 1：RLHF 的优化目标是最大化奖励同时约束与参考策略的 KL 散度：

$$\max_{\pi_\theta} \mathbb{E}_{y\sim\pi_\theta}[r(x,y)] - \beta \cdot D_{KL}[\pi_\theta || \pi_{ref}]$$

Step 2：这个约束优化问题有封闭形式的解：

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{ref}(y \mid x)\exp(\frac{r(x,y)}{\beta})$$

Step 3：反解出奖励函数：

$$r(x,y) = \beta\log\frac{\pi^*(y \mid x)}{\pi_{ref}(y \mid x)} + \beta\log Z(x)$$

Step 4：代入 Bradley-Terry 偏好模型 $P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$，配分函数 $Z(x)$ 被消掉：

$$P(y_w \succ y_l \mid x) = \sigma(\beta[\log\frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \log\frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)}])$$

关键：奖励函数 r 被策略模型和参考模型的对数概率比替代了，不再需要显式的奖励模型。

MiniMind 中的实现：

MiniMind 的 `train_dpo.py` 直接实现了这个简化后的损失：

```
chosen_rewards = beta * (policy_chosen_logps - ref_chosen_logps)
rejected_rewards = beta * (policy_rejected_logps - ref_rejected_logps)
```

```
loss = -F.logsigmoid(chosen_rewards - rejected_rewards).mean()
```

$\beta = 0.1$

，参考模型是 SFT 后的模型（冻结参数）。

追问方向： - DPO 的推导中，为什么 $Z(x)$ 能被消掉？ - 如果偏好数据噪声很大，DPO 的推导还成立吗？

Q31: DPO 的 loss 函数公式是什么？

标准答案：

$$\mathcal{L}_{DPO}(\pi_\theta;\pi_{ref}) = -\mathbb{E}_{(x,y_w,y_l)\sim D}[\log\sigma(\beta[\log\frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \log\frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)}])]$$

逐符号解释： - π_θ：当前训练的策略模型 - π_{ref}：冻结的参考模型（SFT checkpoint） - y_w：chosen 回答（人类偏好的） - y_l：rejected 回答（人类不偏好的） - β：温度参数，控制偏离参考模型的程度 - σ：sigmoid 函数 - D：偏好数据集

直觉理解：定义隐含奖励差 $\Delta = \beta[\log\frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \log\frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)}]$。优化目标就是让 Δ 尽可能大——让模型相比参考模型更多地提升好回答的概率、降低坏回答的概率。

MiniMind 中的实现：

log 概率计算：$\log\pi_\theta(y \mid x) = \sum_{t=1}^{T} \log\pi_\theta(y_t \mid x, y_{<t})$

```
def get_log_probs(model, input_ids, labels):
    logits = model(input_ids).logits
    log_probs = F.log_softmax(logits, dim=-1)
    per_token_logps = torch.gather(log_probs, 2, labels.unsqueeze(2)).squeeze(2)
    return per_token_logps.sum(dim=-1)
```

使用 `dpo.jsonl` 数据集，包含 prompt、chosen、rejected 三字段。

追问方向： - DPO 的 loss 和二分类的 BCE loss 有什么关系？ - 如果 chosen 和 rejected 的质量差异很小，训练效果会怎样？

Q32: PPO 的核心思想是什么？clip 机制的作用？

标准答案：

PPO（Proximal Policy Optimization） 的核心思想是在最大化期望奖励的同时，限制每次策略更新的幅度，防止"步子太大扯到蛋"。

PPO-Clip 目标函数：

$\mathcal{L}_{PPO} = -\mathbb{E}[\min(r_t \cdot A_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon) \cdot A_t)]$

其中 $r_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{old}(a_t \mid s_t)}$ 是新旧策略的概率比，A_t 是优势函数，ε 通常取 0.2。

clip 的作用：- 当 $A_t > 0$（好动作）：r_t 被截断在 $1+\varepsilon$，防止过度增加概率 - 当 $A_t < 0$（坏动作）：r_t 被截断在 $1-\varepsilon$，防止过度降低概率 - 本质是近似实现 Trust Region：每次更新只在旧策略附近的"信任区域"内进行

为什么需要 clip？ 原始策略梯度如果 r_t 特别大，梯度爆炸，训练崩溃。TRPO 用 KL 约束但计算昂贵；PPO 的 clip 是简单高效的替代。

MiniMind 中的实现：

MiniMind 的 `train_rl.py` 实现了 PPO/GRPO 的 clip 机制，$\varepsilon = 0.2$。完整损失还包括价值函数损失和 KL 惩罚：

$\mathcal{L} = \mathcal{L}_{PPO} + c_1 \cdot \mathcal{L}_{VF} + c_2 \cdot \mathcal{L}_{entropy} + \beta \cdot D_{KL}$

追问方向： - PPO-Clip 和 PPO-Penalty（KL 惩罚版本）哪个更常用？ - clip 范围 ε 对训练稳定性的影响？

Q33: PPO 需要哪四个模型？各自的作用是什么？

标准答案：

PPO 训练需要同时维护**四个模型**：

模型	作用	是否更新
Policy Model（策略模型）	生成回答，是要优化的目标模型	✅ 更新
Reference Model（参考模型）	SFT 后的模型，作为 KL 惩罚的锚点	❌ 冻结
Reward Model（奖励模型）	对生成的回答打分	❌ 冻结
Value Model（Critic 网络）	估计每个状态的期望奖励，用于计算优势函数	✅ 更新

为什么需要四个模型？ - Policy Model：核心目标，学习生成更好的回答 - Reference Model：防止策略退化，保持基本语言能力 - Reward Model：提供奖励信号，评估回答质量 - Value Model：用于计算 GAE 优势函数，减小策略梯度的方差

这四个模型带来巨大的显存和计算开销，是 PPO 训练复杂的主要原因。

MiniMind 中的实现：

MiniMind 使用 InternLM2-1.8B-Reward 作为奖励模型（1.8B 参数，单卡可运行），属于 RLAIF。策略模型和参考模型都是 MiniMind 的 SFT checkpoint。GRPO 方案去掉了 Value Model，只需三个模型。

追问方向： - 能否用同一个模型同时做 Policy 和 Value（共享参数）？ - 如果奖励模型本身不准确，PPO 训练会出什么问题？

Q34: GAE（Generalized Advantage Estimation）是什么？

标准答案：

GAE 是一种用于计算优势函数的方法，通过在偏差（bias）和方差（variance）之间权衡来获得更平滑的估计。

优势函数定义：$A_t = R_t - V(s_t)$，表示动作 a_t 比平均水平好多少。

GAE 公式：

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}$$

其中 TD 误差 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$。

参数含义： - γ：折扣因子（0.99），控制对未来奖励的重视程度 - λ：GAE 系数（0.95），控制偏差-方差权衡 - $\lambda = 0$：$A_t = \delta_t$（TD(0)），低方差高偏差 - $\lambda = 1$：$A_t = R_t - V(s_t)$（Monte Carlo），高方差低偏差 - $\lambda \in (0, 1)$：折中，通常取 0.95

为什么不直接用 R_t？

直接用累积奖励 R_t 方差极大，训练不稳定。减去基线 $V(s_t)$ 是方差缩减技术，GAE 进一步通过指数加权平滑了优势估计。

MiniMind 中的实现：

MiniMind 的 PPO 实现中使用 GAE 计算优势函数。由于 LLM 场景中奖励通常只在序列结束时给出（稀疏奖励），中间步的 $r_t = 0$，Critic 网络负责估计中间状态的价值。

追问方向： - 在 LLM 场景下，GAE 的 γ 和 λ 通常取什么值？ - 为什么说 Critic 网络在 LLM 场景下估计不准？

Q35: GRPO 是什么？它和 PPO 有什么区别？

标准答案：

GRPO（Group Relative Policy Optimization） 是 DeepSeek 提出的方法，核心创新是**用组内相对排名替代 Critic 网络来估计优势函数**，完全不需要 Value Model。

核心流程： 1. 对同一个 prompt，采样 G 个回答 $\{y_1, y_2, \ldots, y_G\}$ 2. 用奖励模型对每个回答打分 $\{r_1, r_2, \ldots, r_G\}$ 3. 组内标准化得到优势估计：$\tilde{A}_i = \frac{r_i - \text{mean}(r_{1..G})}{\text{std}(r_{1..G})}$ 4. 用类似 PPO 的 clip 损失优化策略 + 显式 KL 惩罚

GRPO vs PPO：

维度	PPO	GRPO
Critic 网络	需要	**不需要**
优势计算	GAE + Critic	组内归一化奖励
内存	4 个模型	3 个模型
采样	每 prompt 1 条回答	每 prompt G 条回答
KL 正则	隐式（clip）	显式 $\beta \cdot D_{KL}$
适用场景	通用 RL	更适合 LLM 场景

MiniMind 中的实现：

MiniMind 的 `train_rl.py` 实现了 GRPO，每个 prompt 生成 G 个回答：

```
def group_normalize(rewards, group_size):
    rewards = rewards.view(-1, group_size)
    mean = rewards.mean(dim=1, keepdim=True)
    std = rewards.std(dim=1, keepdim=True)
    advantages = (rewards - mean) / (std + 1e-8)
    return advantages.view(-1)
```

使用 InternLM2-1.8B-Reward 打分，KL 散度通过 `policy_logprobs - ref_logprobs` 显式计算。

追问方向： - G（组大小）如何选择？G=2 和 G=16 有什么区别？ - GRPO 的组内标准化和 PPO 的 GAE 哪个优势估计更准确？

Q36: GRPO 如何用组内奖励归一化替代 Critic 网络?

标准答案:

传统 PPO 的 Critic 困境:

在 LLM 场景中，Critic 网络面临严峻挑战： 1. 奖励通常只在序列结束时给出（稀疏奖励），中间状态的价值很难估计 2. 状态空间极大（所有可能的 token 序列），泛化困难 3. 需要额外训练一个和 Policy 同等规模的模型

GRPO 的替代方案:

核心洞见——不需要精确估计每个状态的绝对价值，只需要知道**同一个 prompt 的多个回答中哪个相对更好**。

具体做法： 1. 对同一 prompt 采样 G 个回答 2. 奖励模型打分得到 $\{r_1, ..., r_G\}$ 3. 组内标准化：$\tilde{A}_i = \frac{r_i - \mu}{\sigma}$，其中 μ, σ 是组内均值和标准差 4. $\tilde{A}_i > 0$ 的回答被鼓励，$\tilde{A}_i < 0$ 的被抑制

为什么这样做有效？ - 同一 prompt 的不同回答共享相同的"难度基线"，组内比较消除了 prompt 难度差异 - 标准化后的优势是零均值的，天然具有正负信号 - 不需要训练任何额外网络，直接使用奖励模型的输出

MiniMind 中的实现:

```
rewards = reward_model.score(prompts, responses)  # [B*G]
advantages = group_normalize(rewards, G)           # 组内标准化
```

标准化使用 `(rewards - mean) / (std + 1e-8)`，其中 `1e-8` 防止除零。

追问方向: - 如果组内所有回答的奖励都很接近（std ≈ 0），会出什么问题？ - 组内标准化和 Batch Normalization 有什么联系？

Q37: CISPO 是什么？它解决了 GRPO 的什么问题?

标准答案:

CISPO（Confidence Interval based Safe Policy Optimization） 是一种基于置信区间的安全策略优化方法，与 PPO 的 clip 机制相比，使用了更精细的策略约束方式。

PPO clip 的问题: - clip 范围 $[1 - \varepsilon, 1 + \varepsilon]$ 是固定的，对所有 token 一视同仁 - 不考虑采样数据的统计可靠性——数据稀疏时仍然使用相同的约束强度

CISPO 的改进： 1. 根据采样数据的统计特性，计算策略更新的**置信区间** 2. 只在置信区间内更新策略，超出部分**自适应收缩** 3. 数据充足时允许更大幅度的更新，数据稀疏时自动收紧约束 4. 理论上比 clip 更精细，因为约束强度随数据自适应调整

在 PO 统一框架中的位置：

算法	策略约束方式	正则项
PPO	clip$(r_t, 1-\varepsilon, 1+\varepsilon)$	$\beta \cdot KL$
CISPO	**置信区间约束**	**自适应**

MiniMind 中的实现：

MiniMind 的 L18 文档中简要介绍了 CISPO，与 PPO/GRPO 同属 PO 算法族。具体实现细节较少公开，但核心思想是用统计置信区间替代固定 clip 范围。

追问方向： - 置信区间是如何计算的？基于什么统计假设？ - CISPO 在小模型（如 MiniMind 64M）上效果如何？

Q38: 偏好优化算法的统一框架 $\mathcal{L}_{PO} = \mathbb{E}[f(r_t) \cdot g(A_t) - h(KL_t)]$ 是什么意思？

标准答案：

这个统一框架将所有 PO（Policy Optimization）算法分解为三个正交的组件：

1. 策略项 $f(r_t)$：概率比 $r_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{old}(a_t \mid s_t)}$ 的函数，控制"策略更新的方向和幅度"。 - PPO/GRPO：min $(r_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon))$ - DPO：隐式（概率比被吸收进 loss 公式） - CISPO：置信区间约束的 r_t

2. 优势项 $g(A_t)$：优势函数的变换，描述"这个动作有多好"。 - PPO：A_t（由 GAE + Critic 计算） - GRPO：$\tilde{A}_t$（组内归一化奖励） - DPO：隐式偏好（chosen vs rejected 的对比）

3. 正则项 $h(KL_t)$：KL 散度惩罚，防止策略跑得太远。 - PPO：$\beta \cdot KL$（有些实现通过 clip 隐式约束） - GRPO：$\beta \cdot D_{KL}(\pi_\theta || \pi_{ref})$（显式） - DPO：隐式（内嵌在 loss 中） - CISPO：自适应 KL

框架的价值：理解这个统一视角后，可以自由组合不同的策略约束、优势估计和正则化方法来设计新的 PO 算法。

MiniMind 中的实现：

MiniMind 的 L18 文档明确给出了这个统一框架，并将 PPO、GRPO、DPO、CISPO 都纳入其中。实际实现的 `train_rl.py` 使用 GRPO 的组合：f = clip, g = 组内归一化, h = 显式 KL 惩罚。

追问方向： - 能否设计一个混合使用 GAE 和组内归一化的优势估计？ - 正则项用 reverse KL 和 forward KL 有什么区别？

Q39: 小模型做 RL 面临哪些挑战？奖励稀疏问题如何解决？

标准答案：

小模型 RL 的特殊挑战：

1. **生成质量不稳定**：64M 参数的小模型在探索时可能生成大量低质量、不连贯的回答
2. **奖励信号噪声大**：小模型回答方差大，奖励分数波动剧烈，有效梯度信号弱
3. **优化效率低**：大量探索浪费在低质量区域，有效学习信号稀少
4. **容量瓶颈**：小模型参数有限，难以同时维持语言能力和学习新的对齐行为

奖励稀疏问题：奖励模型通常在整个回答生成完毕后才给出一个总分数，模型无法知道 200 个 token 中哪些贡献了正奖励、哪些贡献了负奖励。

缓解策略： 1. **KL 惩罚**：防止模型偏离 SFT 策略太远（$\beta \cdot D_{KL}$） 2. **GRPO 的组内排名**：降低绝对奖励值的影响，关注相对好坏 3. **减小学习率**：小模型更容易崩溃 4. **从好的 SFT checkpoint 开始**：确保初始策略已经有基本的生成质量 5. **过程奖励模型（PRM）**：在推理的每一步给出奖励，缓解稀疏问题 6. **奖励塑形（Reward Shaping）**：设计中间奖励信号

MiniMind 中的实现：

MiniMind 使用 GRPO（组内排名降低噪声）+ InternLM2-1.8B-Reward（专业奖励模型）+ KL 惩罚 + 从 SFT checkpoint 起步，综合缓解小模型 RL 训练的困难。

追问方向： - 过程奖励模型（PRM）和结果奖励模型（ORM）有什么区别？ - 如果奖励模型给出的分数全是正的，RL 训练会怎样？

Q40: Agentic RL 是什么？多轮 Tool-Use 场景下的 RL 如何训练？

标准答案：

Agentic RL 是指将 LLM 作为 Agent，在多轮交互环境中通过 RL 进行训练。与单轮对话 RL 不同，Agent 需要： 1. 根据环境反馈做出多步决策 2. 正确调用工具（Tool-Use / Function Calling） 3. 根据工具返回的结果继续推理

多轮 Tool-Use 的 RL 训练挑战： 1. **信用分配更难**：奖励可能延迟到多轮交互结束，中间步骤的贡献更难评估 2. **动作空间复杂**：除了生成文本，还要选择工具、构造参数 3. **环境交互代价高**：每轮都需要实际调用工具/API 4. **状态空间更大**：包含整个对话历史和工具返回

训练方法： - 在 SFT 阶段先用标注好的 Tool-Use 数据训练基本能力 - RL 阶段用环境反馈（如工具调用成功率、最终任务完成度）作为奖励 - 可以使用 ReAct（Reasoning + Acting）框架组织思考和行动 - 奖励设计：综合考虑工具调用正确性、最终答案质量、效率（步数）

MiniMind 中的实现：

MiniMind 的 SFT 数据中已混入 Tool Call（函数调用）数据，使模型具备基本的工具调用能力。这为后续的 Agentic RL 训练奠定了基础。目前 MiniMind 的 RL 训练主要聚焦在单轮对话对齐上，多轮 Agent RL 是未来扩展方向。

追问方向： - ReAct 框架中"思考"和"行动"是如何交替进行的？ - 如何设计多轮 Agent 场景下的奖励函数？

Q41: DPO vs PPO vs GRPO 各自的优缺点对比？

标准答案：

维度	DPO	PPO	GRPO
复杂度	低（类似 SFT）	高（4 个模型交互）	中（3 个模型）
模型数量	2（policy + ref）	4（policy + ref + RM + critic）	3（policy + ref + RM）
是否需要 RM	否	是	是
数据类型	Off-Policy（离线偏好对）	On-Policy（在线采样）	On-Policy（在线采样）
训练稳定性	较好	需要精心调参	较好
探索能力	无	强	强
计算开销	低	高	中

维度	DPO	PPO	GRPO
适用场景	偏好对齐、安全性	能力提升、复杂推理	LLM 对齐（兼顾效率和效果）
核心优势	简单、无需 RM	在线探索、理论完备	无需 Critic、内存友好
核心劣势	不做探索、分布偏移	4 个模型、训练不稳定	需要多次采样

选择建议： - 快速对齐 / 资源有限 → **DPO** - 追求最强效果 / 能力提升 → **PPO** - LLM 场景 / 平衡效率和效果 → **GRPO**

MiniMind 中的实现：

MiniMind 三种方法都实现了：`train_dpo.py`（DPO）、`train_rl.py`（PPO/GRPO）。DPO 使用 `dpo.jsonl` 偏好对数据，PPO/GRPO 使用 InternLM2-1.8B-Reward 作为奖励模型，GRPO 每个 prompt 采样 G 个回答做组内比较。

追问方向： - 能否先 DPO 再 PPO/GRPO？两者可以级联使用吗？ - 在什么场景下 GRPO 比 PPO 效果更好？

Q42: 什么是 Reference Model？为什么需要冻结它？

标准答案：

Reference Model（参考模型，π_{ref}） 是 DPO/PPO/GRPO 等对齐算法中的核心组件，通常是 SFT 训练完成后的模型，在对齐训练过程中**完全冻结**参数。

为什么需要 Reference Model？

1. **防止模型退化**：没有参考模型约束，策略模型可能为了获得高奖励/满足偏好而输出怪异、重复或无意义的文本（reward hacking）
2. **保持语言能力**：确保模型在对齐过程中不会丢失基本的语言生成能力和通用知识
3. **KL 惩罚的锚点**：$D_{KL}(\pi_\theta || \pi_{ref})$ 约束策略不偏离参考太远，参考模型就是那个"锚"
4. **DPO 的数学必要**：DPO loss 中 $\log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$ 需要参考模型的概率

为什么必须冻结？ - 如果参考模型也在更新，KL 约束的"锚"就在移动，约束失效 - 冻结保证了 π_{ref} 是一个固定的基线 - 冻结后不需要计算梯度，节省显存

MiniMind 中的实现：

```
ref_model = MiniMindLM.from_pretrained(sft_checkpoint)
ref_model.eval()  # 切换到评估模式
```

```
for param in ref_model.parameters():
    param.requires_grad = False  # 冻结所有参数
```

DPO 和 GRPO 训练中，reference model 只在前向传播中用于计算 $\log \pi_{ref}(y \mid x)$，不参与反向传播。

追问方向： - 如果参考模型不是 SFT 后的模型，而是预训练模型，效果会怎样？ - 有没有不需要参考模型的对齐方法？（如 SimPO、ORPO）

Q43: 白盒蒸馏和黑盒蒸馏的区别是什么？

标准答案：

维度	黑盒蒸馏	白盒蒸馏
访问教师内部	否（只看输出文本）	是（需要 logits 分布）
信息量	少（只有硬标签）	多（完整概率分布，含暗知识）
实现方式	用大模型生成数据，然后做 SFT	学生模型匹配教师的 logits 分布
实现难度	低	高（需要同时运行两个模型）
适用场景	教师模型只能通过 API 访问	教师模型可以本地运行
效果	好	通常更好
显存要求	低（不需要同时加载两个模型）	高（同时加载教师和学生）

黑盒蒸馏的本质：用大模型生成高质量数据，训练小模型。大模型的知识通过生成的文本间接传递。

白盒蒸馏的核心：学生不仅学习正确答案（硬标签），还学习教师的"思考方式"——对每个 token 的完整概率分布（软标签）。软标签中的"暗知识"（dark knowledge）包含了 token 间的相似性关系。

MiniMind 中的实现：

MiniMind 同时使用了两种蒸馏策略： - **黑盒蒸馏（隐式）**： SFT 数据中的高质量回答来自 Qwen3、DeepSeek R1 等强模型 - **白盒蒸馏（显式）**： `train_distillation.py` 使用教师模型的 logits 分布，通过 KL 散度对齐学生和教师

追问方向： - 暗知识（dark knowledge）具体包含哪些信息？ - 黑盒蒸馏和白盒蒸馏能否同时使用？

Q44: 知识蒸馏的 Loss 公式 $\mathcal{L} = \alpha \cdot \text{CE} + (1 - \alpha) \cdot T^2 \cdot \text{KL}$ 如何理解?

标准答案:

$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{CE}} + (1 - \alpha) \cdot T^2 \cdot \mathcal{L}_{\text{KL}}$

逐项解释:

1. 硬标签损失 $\mathcal{L}_{\text{CE}}$: - 标准交叉熵，学生预测 vs 真实标签 - 确保学生模型学会正确答案 - α 控制硬标签的权重

2. 软标签损失 $\mathcal{L}_{\text{KL}}$: - 学生分布 vs 教师分布的 KL 散度 - 使用温度 T 软化后的 softmax: $p_i^T = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$ - 让学生学习教师的"思考方式"

3. 为什么乘以 T^2?: - 温度 T 会让 softmax 的梯度缩小 $1 / T^2$ 倍（logits 除以 T，反向传播经过两次除以 T 的操作） - 乘以 T^2 补偿这个缩放，使硬标签和软标签的梯度量级一致

4. α 的选择: - $\alpha \to 1$: 更依赖硬标签，蒸馏效果减弱 - $\alpha \to 0$: 更依赖软标签，可能不稳定 - 常用值: $\alpha = 0.1 \sim 0.5$

MiniMind 中的实现:

MiniMind 的 `train_distillation.py` 使用 $T = 2.0, \alpha = 0.3$:

```
hard_loss = F.cross_entropy(student_logits, labels, ignore_index=-100)
student_soft = F.log_softmax(student_logits / T, dim=-1)
teacher_soft = F.softmax(teacher_logits / T, dim=-1)
soft_loss = F.kl_div(student_soft, teacher_soft, reduction='batchmean')
loss = alpha * hard_loss + (1 - alpha) * T * T * soft_loss
```

教师模型冻结（`eval()` + `requires_grad=False`），可以用半精度（`.half()`）节省显存。

追问方向: - 如果只用软标签损失（$\alpha = 0$），效果会怎样? - $\mathcal{L}_{\text{KL}}$ 中用的是 $\text{KL}(p_T || p_S)$ 还是 $\text{KL}(p_S || p_T)$? 有区别吗?

Q45: 温度 T 在蒸馏中的作用是什么?

标准答案:

温度 T 用于软化教师模型的输出分布，使"暗知识"显现。

带温度的 Softmax：

$$p_i^T = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

温度的影响： | 温度 T | 分布特点 | 信息量 | |--------|---------|--------| | T = 1 | 尖锐，接近 one-hot | 少，类似硬标签 | | T = 2~5 | 适度平缓 | 丰富，暗知识显现 | | $T \to \infty$ | 趋近均匀分布 | 过度平滑，失去区分度 |

为什么需要高温度？

当 T = 1 时，教师输出可能是 [0.95, 0.03, 0.01, 0.01]，接近硬标签，学生学不到什么额外信息。

当 T = 3 时，变为 [0.50, 0.25, 0.15, 0.10]——学生能学到： - "狗"比"鱼"更接近"猫"（暗知识） - 各个候选之间的相对关系 - 教师对不确定性的判断

类比：教师判卷——低温只说"A 对其他都错"；高温说"A 最对（70分）、B 有道理（20分）、C 部分对（8分）、D 完全错（2分）"。

注意：教师和学生的 softmax 必须使用**相同的** T。

MiniMind 中的实现：

MiniMind 使用 T = 2.0，这是一个常用的温度值，在信息丰富度和分布稳定性之间取得平衡。

追问方向： - T 太大会出什么问题？（分布过于平缓，失去区分度） - 有没有自适应温度的蒸馏方法？

Q46: MiniMind 如何从 Qwen 系列进行知识蒸馏？

标准答案：

MiniMind 从 Qwen 系列模型进行知识蒸馏采用了**双重策略**：

1. 黑盒蒸馏（隐式/数据级）： - SFT 训练数据中的高质量回答由 Qwen3、DeepSeek R1 等强模型生成 - 这些数据本身就是黑盒蒸馏的产物——大模型的知识通过生成文本间接传递给小模型 - 不需要访问大模型的内部参数，只需要其输出文本 - 14GB 的 SFT 数据中包含大量由强模型生成的知识

2. 白盒蒸馏（显式/logits 级）： - `train_distillation.py` 实现了完整的白盒蒸馏 - 使用 Qwen 系列模型作为教师模型，获取其 logits 分布 - 学生模型（MiniMind）通过 $\mathcal{L} = \alpha \cdot \text{CE} + (1-\alpha) \cdot T^2 \cdot \text{KL}(p_T || p_S)$ 学习匹配教师分布 - 教师模型冻结，不参与反向传播 - 需要同时加载教师和学生模型

显存管理：

```
teacher_model.eval()
for param in teacher_model.parameters():
    param.requires_grad = False
teacher_model.half()  # 半精度节省显存
```

MiniMind 中的实现：

典型蒸馏配置：$T = 2.0, \alpha = 0.3$。教师模型的选择需要平衡效果和显存——太大的教师模型可能无法和学生模型同时加载到 GPU。MiniMind 64M 的学生模型很小，可以使用相对较大的 Qwen 模型作为教师。

追问方向： - 教师模型和学生模型的词表不一致时怎么处理？ - 蒸馏后的学生能超越教师吗？（通常不能，但在特定子任务上可能例外）

Q47: MoE 混合专家模型的原理是什么？

标准答案：

MoE（Mixture of Experts） 是一种稀疏激活（Sparse）模型架构：用多个并行的 FFN "专家"替代标准 Transformer 中的单个 FFN，通过 Router（路由器）决定每个 token 送给哪些专家处理。

核心原理： 1. **每个专家**是一个独立的 FFN 网络（结构相同，参数不同） 2. **Router** 是一个简单的线性层 + Softmax：`nn.Linear(hidden_dim, num_experts)` 3. 每个 token 经过 Router 后得到对每个专家的"偏好分数" 4. 选择分数最高的 Top-K 个专家处理该 token 5. 最终输出是被选中专家输出的加权和

Dense vs Sparse： - Dense 模型：每个 token 经过所有参数，总参数量 = 激活参数量 - MoE（Sparse）：每个 token 只激活部分专家，总参数量 >> 激活参数量

核心优势：在不增加每 token 计算量的前提下扩大模型容量。

MiniMind 中的实现：

MiniMind-3-MoE 只修改了 FFN 层（Attention 层不变），使用 `MoEGate`（路由器）+ `MOEFeedForward`（多专家 FFN）：

```
class MoEGate(nn.Module):
    def __init__(self, config):
        self.gate = nn.Linear(config.dim, config.n_routed_experts, bias=False)
    def forward(self, x):
```

```
        scores = self.gate(x).softmax(dim=-1)
        topk_weight, topk_idx = torch.topk(scores, k=self.top_k, dim=-1)
        return topk_idx, topk_weight, scores
```

追问方向： - MoE 中的每个专家真的会"专业化"吗？ - 为什么只改 FFN 层而不改 Attention 层？

Q48: Top-K 路由策略是什么？负载均衡 Loss 的作用？

标准答案：

Top-K 路由：Router 为每个 token 计算对所有专家的概率分布，然后选择概率最高的 K 个专家处理该 token。

- **Top-1**：每个 token 只送给 1 个专家（计算量最小，MiniMind 使用）
- **Top-2**：送给 2 个专家，输出加权和（更常见，如 Mixtral-8x7B）

选中专家的权重需要归一化：`weights = topk_values / topk_values.sum(dim=-1)`

负载均衡 Loss：

防止"专家塌缩"（所有 token 涌向同一专家）的辅助损失：

$$\mathcal{L}_{balance} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

- N：专家数量
- f_i：实际被路由到专家 i 的 token 比例（频率）
- P_i：Router 对专家 i 的平均概率
- α：平衡系数

直觉：如果某专家同时有高频率（f_i 大）和高概率（P_i 大），loss 就大，梯度推动 Router 分散负载。当所有专家均匀分配时，$f_i = P_i = 1/N$，loss 最小。

MiniMind 中的实现：

MiniMind-MoE 使用 Top-1 路由，负载均衡系数 `router_aux_loss_coef = 5e-4`（$\alpha = 0.01$ 量级）。总训练损失：

$$\mathcal{L}_{total} = \mathcal{L}_{LM} + \alpha \cdot \mathcal{L}_{balance}$$

在训练循环中：`loss = res.loss + res.aux_loss`。

追问方向： - α 太大或太小分别有什么影响？ - 除了负载均衡 loss，还有什么方法防止专家塌缩？（如 random routing、auxiliary token dropping）

Q49: MoE 训练中的专家坍缩问题是什么？如何解决？

标准答案：

专家坍缩（Expert Collapse） 是 MoE 训练中最大的隐患：所有 token 都涌向同一个（或少数几个）专家，其他专家得不到训练，实质上退化为 Dense 模型。

坍缩的原因——正反馈循环： 1. 某个专家一开始碰巧表现稍好 2. Router 将更多 token 路由给它 3. 它得到更多训练数据，表现更好 4. 更多 token 被路由过来…… 5. 最终其他专家几乎不被使用

坍缩的后果： - 模型有效参数量大幅缩减（4 个专家只用了 1 个 ≈ 浪费了 75% 参数） - 性能显著低于正常 MoE - 训练资源浪费

解决方案： 1. **负载均衡 Loss**：$\mathcal{L}_{balance} = \alpha \cdot N \cdot \sum f_i \cdot P_i$（最常用） 2. **随机路由**：在 Top-K 选择中加入随机性，让低概率专家也有机会被选中 3. **Capacity Factor**：限制每个专家处理的 token 数量上限，超出的 token 被丢弃或重分配 4. **Expert Choice**：不让 token 选专家，而让专家选 token（Switch Transformer） 5. **初始化策略**：确保 Router 初始化时对各专家的权重接近均匀

MiniMind 中的实现：

MiniMind-MoE 使用负载均衡 Loss 防止坍缩，系数 `router_aux_loss_coef = 5e-4`。Router 的 gate 层使用无 bias 的线性层（`bias=False`），Softmax 归一化后通过 Top-K 选择。训练日志中可以监控各专家的使用频率来检测是否发生坍缩。

追问方向： - 如何在训练日志中监控专家使用分布？ - Expert Choice 和 Token Choice 的优劣对比？

Q50: MiniMind-MoE 的配置是什么？为什么选择 4 个专家 top-1？

标准答案：

MiniMind-3-MoE 配置：

配置项	值
专家数量（`n_routed_experts`）	4
Top-K（`num_experts_per_tok`）	1

配置项	值
总参数量	198M
激活参数量	64M（≈ MiniMind Dense 版）
隐藏维度	与 Dense 版相同（768）
FFN 维度	与 Dense 版相同（~2048）
层数	与 Dense 版相同（8）
负载均衡系数	5e-4

参数量计算：

```
Dense 版：Embedding + Attention + 1×FFN ≈ 64M
MoE 版：Embedding + Attention + 4×FFN ≈ 198M（FFN 部分 ×4）
Top-1 激活：Embedding + Attention + 1×FFN ≈ 64M
```

为什么选择 4 个专家 Top-1？

1. **教学可行性**：4 个专家是最小的有意义 MoE 配置，易于理解和调试
2. **显存可控**：198M 总参数仍可在单卡（24GB）上训练
3. **激活参数量对齐**：Top-1 时激活参数 ≈ Dense 版的 64M，便于公平对比
4. **训练效率**：专家数量越多，负载均衡越难，训练越不稳定；4 个专家是稳定性和多样性的平衡点
5. **Top-1 vs Top-2**：Top-1 计算量更小，在专家较少时已经足够，且代码实现更简单

效果对比： - 相同激活参数量（64M）：MoE 版通常优于 Dense 版（更大参数容量） - 相同总参数量：Dense 版通常更好（每个 token 用到了全部参数） - 训练速度：MoE 在原生 PyTorch 上比 Dense 慢（kernel 调度开销）

MiniMind 中的实现：

MiniMind-MoE 的配置通过 `MiniMindConfig` 设置：`use_moe=True, n_routed_experts=4, num_experts_per_tok=1`。通过 `router_aux_loss_coef=5e-4` 控制负载均衡 loss 的强度。每个专家的 FFN 结构使用与 Dense 版相同的 SwiGLU 激活函数。

追问方向： - 如果将专家数量增加到 8 个 Top-2，参数量和激活参数量分别变成多少？ - MoE 模型在推理时能否只加载被选中的专家以节省显存？